SISTEMAS DIGITALES

Apuntes de Clase 10 — Anatomía de la ISR, Atributo IRAM, Variable Volatile, Patrón Bandera y Antirrebote

Nota académica: Apuntes digitalizados directamente correspondientes a la Clase 9. De mi autoría

1. Anatomía y Reglas de Oro de una ISR

Una **ISR (Interrupt Service Routine)** es una función del sistema que el programador nunca invoca directamente; su ejecución es disparada de manera asíncrona por el hardware del ESP32. Su regla de oro fundamental es **ser extremadamente corta** (como apagar un fuego con un extintor: actuar rápido y salir).

SÍ Permitido en una ISR	PROHIBIDO en una ISR
Subir una variable bandera (flag)	Usar funciones de comunicación serie (Serial.print / println)
Guardar marcas de tiempo con millis() / micros()	Ejecutar retardos (delay() / delayMicroseconds())
Incrementar contadores numéricos simples	Realizar cálculos matemáticos complejos o bucles de espera

El Perro Guardián (Watchdog): Si una ISR tarda demasiado tiempo ejecutándose, el sistema operativo interpreta que el núcleo está bloqueado y dispara el mecanismo de seguridad del **Watchdog**, reiniciando abruptamente el ESP32 y generando el conocido error `'Guru Meditation Error'`.

2. Alojamiento en Memoria Rápida con IRAM_ATTR

El ESP32 almacena los programas grandes en la memoria Flash externa (lenta). Durante ciertas operaciones de escritura en la Flash, la caché del sistema puede desactivarse temporalmente. Si en ese preciso instante ocurre una interrupción y el código de la ISR se encuentra en la Flash, la CPU no podrá leerlo y el sistema colapsará. Por este motivo, **toda ISR debe declararse obligatoriamente con el macro IRAM_ATTR** para asegurar que resida permanentemente en la memoria rápida interna (**Instruction RAM / IRAM**).

3. Sincronización de Datos con volatile

El compilador de C/C++ es optimizador por naturaleza: cuando una variable global es leída repetidamente en el bucle principal sin aparente modificación local, el compilador almacena una copia

en un **registro interno de la CPU** para agilizar el acceso. Sin embargo, como la ISR modifica la variable en la RAM principal de forma asíncrona a espaldas del bucle, la CPU jamás se enterará del cambio si consulta su copia local.

La solución consiste en anteponer la palabra clave `volatile` (ej. `volatile bool alerta = false;`), lo cual le indica explícitamente al compilador que la variable puede cambiar en cualquier momento por acción del hardware, forzando a consultar siempre la RAM principal.

4. El Patrón Bandera (Flag Pattern)

Para delegar con eficiencia el trabajo entre la alta velocidad de la interrupción y el procesamiento general, se implementa el **Patrón Bandera (o Buzón)**:

- ISR (Cartero rápido): Al ocurrir el evento, únicamente levanta la bandera (`alerta = true;`) y finaliza de inmediato.
- Loop (Dueño de casa calmado): En cada vuelta revisa si la bandera está arriba; si es así, la baja de inmediato (`alerta = false;`) y procede a ejecutar las tareas pesadas (impresiones serie, activación de sirenas, conexión Wi-Fi, etc.) con total tranquilidad.

5. Filtrado de Rebote Mecánico con millis()

Los pulsadores mecánicos generan micro-rebotes eléctricos (múltiples flancos espurios en pocos milisegundos al presionar). Si se gestionan mediante interrupciones sin control, un solo toque físico podría disparar la ISR múltiples veces de forma errónea.

```
uint32_t tUltimo = 0;
const uint32_t T_REBOTE = 50; // ms

void IRAM_ATTR isrBoton() {
    uint32_t ahora = millis();
    if (ahora - tUltimo >= T_REBOTE) {
        alerta = true;
        tUltimo = ahora;
    }
}
```

Este algoritmo compara el tiempo actual con el registro `tUltimo`; si el nuevo flanco ocurre antes de que transcurran los 50 milisegundos establecidos, es descartado automáticamente como un rebote mecánico.